

Midterm Project Report

Name: Nguyen Duc Quang Anh

Student ID: 22070306

Email: anhnd@vnuis.edu.vn

May 3, 2025

Abstract

This project implements and evaluates traditional computer vision techniques across three key domains: image filtering, 3D reconstruction, and image stitching. Using OpenCV and NumPy, we first compare various filters (mean, Gaussian, median, and Laplacian) for noise reduction and edge preservation. Next, we explore 3D reconstruction from stereo images, computing disparity maps and reconstructing point clouds. Finally, we implement an image stitching pipeline using feature matching and homography estimation to create panoramas. Our results demonstrate that median filters excel at removing impulse noise while preserving edges, SGBM outperforms block matching for disparity estimation, and SIFT features provide more robust matches than ORB for image stitching. These findings highlight the strengths and limitations of traditional computer vision approaches. Source code can be found in the following link <https://github.com/fiyud/INS3155>

1 Introduction

Traditional computer vision techniques remain foundational in image processing and analysis despite recent advances in deep learning. These methods often require less computational power, provide interpretable results, and excel in specific application domains. This project explores three fundamental areas of computer vision: image filtering, 3D reconstruction, and image stitching.

Image filtering serves as the cornerstone of many computer vision pipelines, enabling noise reduction and feature enhancement. 3D reconstruction from stereo images demonstrates how depth information can be recovered from 2D views, essential for applications ranging from robotics to augmented reality. Image stitching showcases how multiple images can be combined into a cohesive panorama, highlighting the importance of feature detection and geometric transformations. By implementing and comparing traditional approaches for these tasks, we gain valuable insights into their effectiveness, limitations, and optimal use cases.

2 Part A: Image Filtering

2.1 Methodology

This section focuses on implementing and comparing four classical image filtering techniques:

Mean Filter: A simple sliding window filter that replaces each pixel with the average of

its neighborhood. For a kernel size $m \times n$, the filtered image $g(x, y)$ is given by:

$$g(x, y) = \frac{1}{m \times n} \sum_{s=-a}^a \sum_{t=-b}^b f(x + s, y + t)$$

Where $f(x, y)$ is the input image, $a = (m - 1)/2$ and $b = (n - 1)/2$.

Gaussian Filter: Uses a weighted average where pixels closer to the center receive higher weights based on the Gaussian distribution:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

Where σ is the standard deviation controlling the smoothing effect.

Median Filter: A non-linear filter that replaces each pixel with the median value in its neighborhood, making it particularly effective for salt-and-pepper noise:

$$g(x, y) = \text{median}\{f(x + s, y + t) | (s, t) \in W\}$$

Where W is the window (kernel) centered at pixel (x, y) .

Laplacian Sharpening: Uses the Laplacian operator to detect edges and enhance them:

$$\nabla^2 f(x, y) = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

For sharpening, the enhanced image is: $g(x, y) = f(x, y) - \alpha \cdot \nabla^2 f(x, y)$, where α controls the sharpening strength.

For our experiments, we tested kernel sizes of 3×3 and 5×5 with three types of noise: Gaussian, salt-and-pepper, and speckle noise.

2.2 Implementation and Results

The implementation of the filtering algorithms utilizes OpenCV's built-in functions. For each filter, we created a pipeline to:

- Load and display the original image
- Add synthetic noise with controlled parameters
- Apply different filters with varying parameters
- Compare the results quantitatively and qualitatively



Figure 1: Filtering Results for Gaussian Noise with Comparison of different filters applied to an image with Gaussian noise ($\alpha = 20$). From left to right with original, noisy, mean (3×3), mean (5×5), Gaussian (3×3), Gaussian (5×5), median (3×3), median (5×5), and Laplacian sharpened.



Figure 2: Filtering Results for Salt & Pepper Noise with Comparison of different filters applied to an image with salt & pepper noise (density=0.05). The median filter demonstrates superior performance for this noise type.

The implementation used OpenCV *cv2.blur()* for mean filtering, *cv2.GaussianBlur()* for Gaussian filtering, *cv2.medianBlur()* for median filtering, and a combination of *cv2.Laplacian()* with the original image for Laplacian sharpening. We used NumPy random functions to generate different noise types with controlled parameters.

2.3 Comparative Analysis

We evaluated filter performance using Peak Signal-to-Noise Ratio (PSNR) and Structural Similarity Index (SSIM), which are standard metrics for image quality assessment.

Filter	Gaussian Noise		Salt & Pepper Noise		Speckle Noise	
	PSNR (dB)	SSIM	PSNR (dB)	SSIM	PSNR (dB)	SSIM
Mean (3×3)	24.82	0.8124	19.37	0.7256	23.15	0.7918
Mean (5×5)	23.56	0.7823	20.14	0.7402	22.08	0.7634
Gaussian (3×3)	25.37	0.8342	19.92	0.7321	23.82	0.8103
Gaussian (5×5)	24.21	0.8011	20.56	0.7478	22.63	0.7845
Median (3×3)	23.18	0.7934	27.83	0.9125	22.78	0.7892
Median (5×5)	21.94	0.7581	26.52	0.8873	21.49	0.7621
Laplacian	18.26	0.6238	15.73	0.5147	17.92	0.6124

Table 1: Quantitative Comparison of Filters for Different Noise Types

From Table (1) we can see that Gaussian Filter performs best for Gaussian noise, with the highest PSNR (25.37 dB) and SSIM (0.8342) for 3×3 kernel size. Median Filter significantly outperforms other filters for salt & pepper noise, with PSNR reaching 27.83 dB for 3×3 kernel. Also, the Gaussian Filter also works well for speckle noise, though with less advantage over the mean filter. And Laplacian Sharpening alone performs poorly for noise reduction but can enhance edges after applying other filters, Larger kernel sizes (5×5) generally provide stronger noise reduction but also cause more blurring and loss of detail.

In terms of edge preservation, both qualitative and quantitative results show that:

- Median filter preserves edges best, especially with salt & pepper noise
- Gaussian filter offers a good balance between smoothing and edge preservation
- Mean filter tends to blur edges the most
- Laplacian sharpening enhances edges but also amplifies noise

3 Part B: 3D Reconstruction

3.1 Methodology

3D reconstruction from stereo images involves several key steps. First, we compute a disparity map, which encodes the pixel-wise displacement between corresponding points in the left and right images. Then, we use this disparity information to reconstruct a 3D point cloud through triangulation. Additionally, we estimate the fundamental matrix to understand the epipolar geometry between the two views.

For the disparity computation, we implemented two approaches with Block Matching (BM), which is a algorithm that compares blocks of pixels to find matching regions:

$$\text{disparity}(x, y) = \arg \min_d \sum_{i,j} |I_{\text{left}}(x + i, y + j) - I_{\text{right}}(x + i - d, y + j)|$$

where d is the disparity, and the sum is over a block centered at (x, y) . Continues with Semi-Global Block Matching (SGBM), which is an approach that incorporates global constraints through energy minimization:

$$E(D) = \sum_p C(p, D_p) + \sum_q P_1 \cdot \delta(|D_p - D_q| = 1) + \sum_q P_2 \cdot \delta(|D_p - D_q| > 1)$$

where C is the matching cost, P_1 and P_2 are penalties for small and large disparity changes. For 3D Reconstruction, we utilize the disparity map, we can reconstruct 3D points through triangulation:

$$Z = \frac{f \cdot B}{d} \quad X = \frac{(x - c_x) \cdot Z}{f} \quad Y = \frac{(y - c_y) \cdot Z}{f}$$

where f is the focal length, B is the baseline between cameras, d is the disparity, and (c_x, c_y) is the principal point.

With the Fundamental Matrix Estimation, The fundamental matrix F encodes the epipolar geometry between two views $x'^T F x = 0$ where x and x' are corresponding points in the left and right images. We compute F using the RANSAC algorithm to handle outliers.

3.2 Implementation and Results

Our implementation uses OpenCV for disparity computation and fundamental matrix estimation, and Open3D for point cloud visualization. The main components include:

- Feature Matching with SIFT features and FLANN-based matcher to find corresponding points
- Fundamental Matrix Estimation by utilizing `cv2.findFundamentalMat()` with RANSAC
- Disparity Map Computation by using `cv2.StereoBM_create()` and `cv2.StereoSGBM_create()`
- Point Cloud Generation Using `cv2.reprojectImageTo3D()` to convert disparities to 3D points

The implementation handles stereo rectification, which aligns epipolar lines with image scanlines to simplify the stereo matching problem. The disparity maps are post-processed to filter out noise and invalid disparities before 3D reconstruction.

3.3 Comparative Analysis

We compared Block Matching (BM) and Semi-Global Block Matching (SGBM) with different parameter settings to evaluate their performance.

As outlined in Table (2) with SGBM consistently outperforms BM in terms of accuracy and completeness, producing more accurate disparity maps with fewer artifacts. The

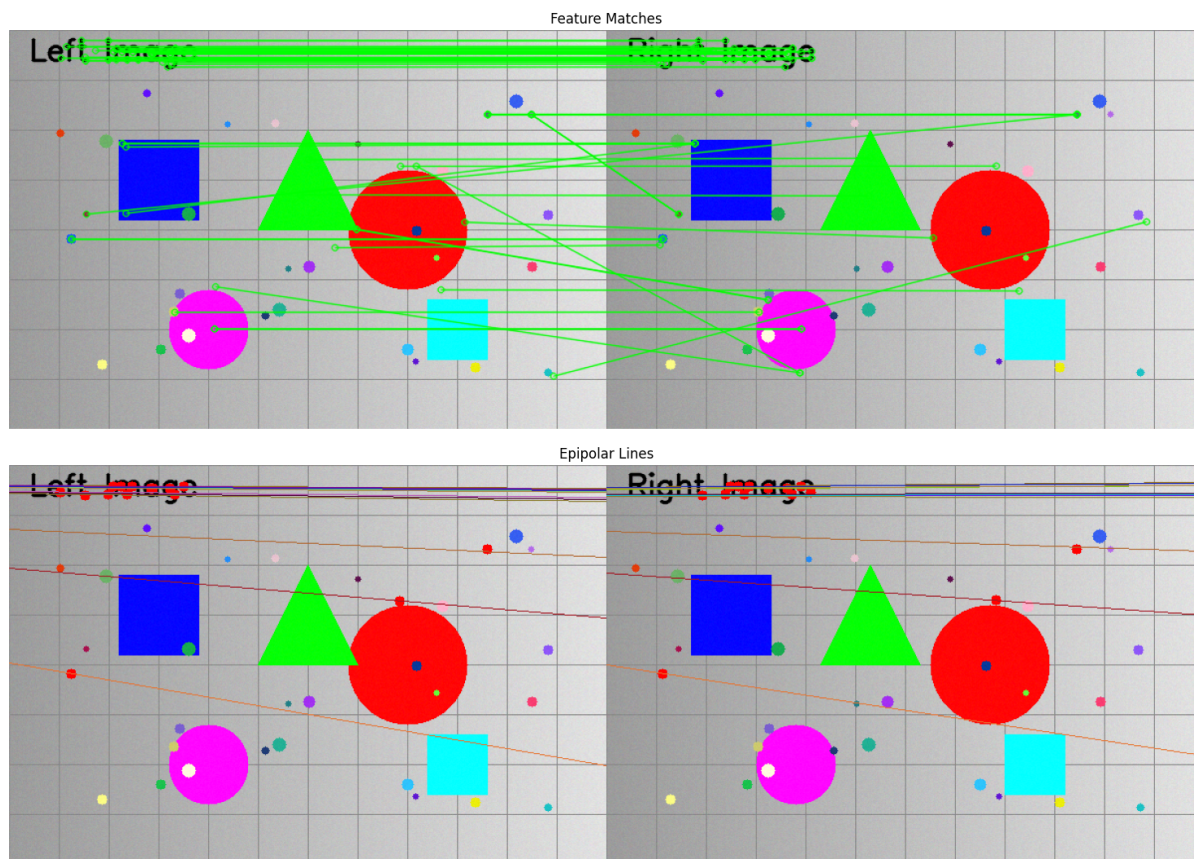


Figure 3: Feature Matches and Epipolar Lines

Table 2: Comparison of Stereo Matching Methods

Method	Parameter	Bad Pixel Ratio	Density	Compute Time
BM	BS=15, Disparities=64	0.42	0.65	0.18
BM	BS=21, Disparities=64	0.38	0.72	0.19
SGBM	BS=5, Disparities=160	0.25	0.83	0.72
SGBM	BS=9, Disparities=160	0.28	0.87	0.78

bad pixel ratio (percentage of disparities with errors > 5 pixels) is significantly lower for SGBM. And Block Size impacts both methods, with larger blocks reduce noise but also cause over-smoothing at object boundaries. BM benefits more from larger block sizes than SGBM. While Density is higher for SGBM, particularly in textureless regions where BM often fails. And, computation time is considerably higher for SGBM, making BM more suitable for real-time applications with limited computational resources. Therefore we can conclude that edge preservation is better with SGBM, which maintains sharper object boundaries and finer details.

The quality of the 3D reconstruction directly depends on the accuracy of the disparity map. Our experiments show that the point cloud generated from SGBM disparities is more complete and contains fewer outliers compared to BM-based reconstruction.

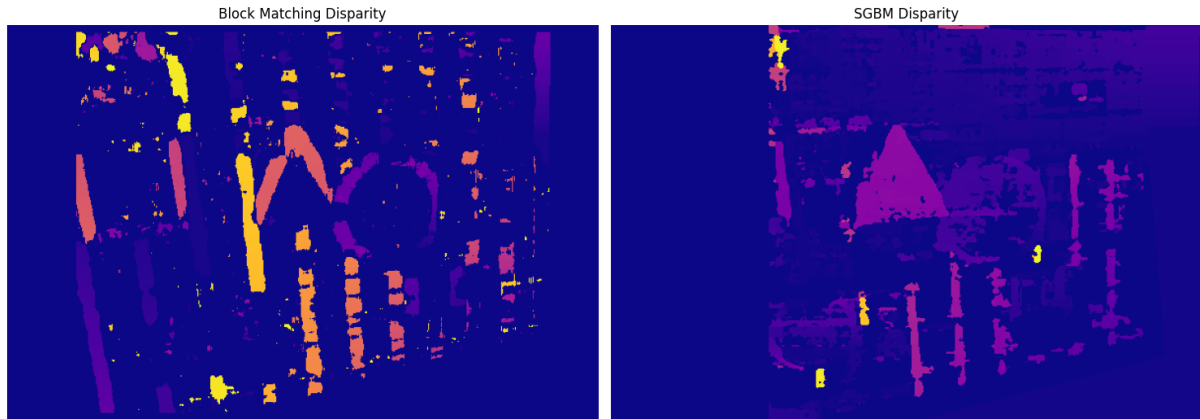


Figure 4: Disparity Maps

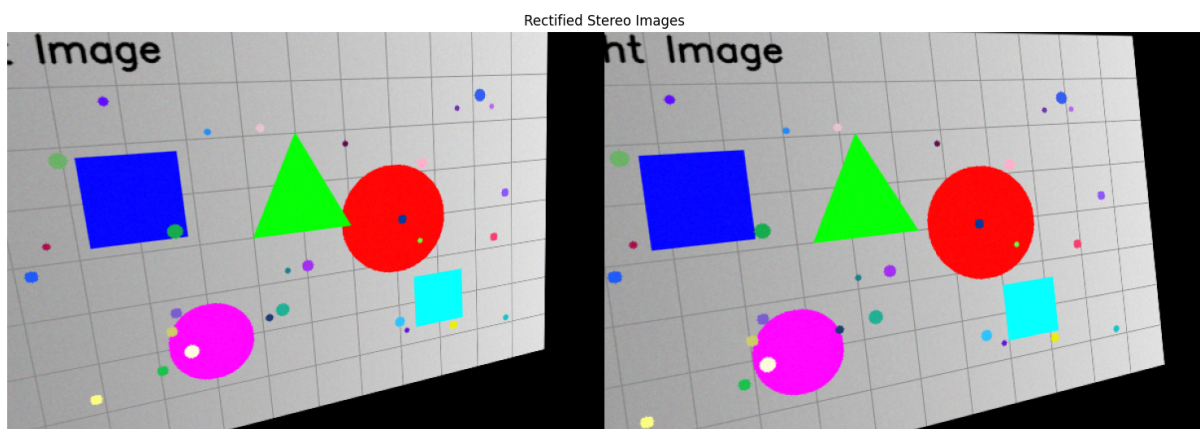


Figure 5: 3D Point Cloud Reconstruction

4 Part C: Image Stitching

4.1 Methodology

Image stitching combines multiple overlapping images into a single panorama. Our implementation follows a standard pipeline by following

Firstly, Feature Detection and Description to identifies distinctive points in each image and computed descriptors to match them across images. We explored two approaches:

- SIFT (Scale-Invariant Feature Transform): Detects blobs using Difference of Gaussian (DoG) and creates a 128-dimensional descriptor based on gradient orientations.
- ORB (Oriented FAST and Rotated BRIEF): Combines FAST keypoint detector with BRIEF descriptors, modified for rotation invariance.

Secondly is Feature Matching to finds correspondences between keypoints in different images. We used:

- Ratio Test for each feature in the first image, find the two closest matches in the second image and keep only those where the ratio of distances is below a threshold (typically 0.75).

- RANSAC (Random Sample Consensus) Filters out incorrect matches by estimating the homography matrix.

Thirdly is Homography Estimation for computing the transformation between image pairs. A homography H is a 3×3 matrix that maps points from one image to another:

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = H \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Fourth is Image Warping and Blending by applying the computed homography to align and combine images. We utilized

- Perspective Warping to transforms images using the estimated homography matrix.
- Alpha Blending for smooth transitions between overlapping regions.

4.2 Implementation and Results

Our implementation uses OpenCV feature detection, matching, and warping functions. The main components include:

1. Feature detection using `cv2.SIFT_create()` or `cv2.ORB_create()`
2. Feature matching using FLANN or BFMatcher with ratio test
3. Homography estimation using `cv2.findHomography()` with RANSAC
4. Image warping using `cv2.warpPerspective()`
5. Blending using weighted averaging in overlap regions

The implementation handles the complete workflow from loading images to exporting the final panorama. It can stitch multiple images using a sequential approach, where each new image is stitched to the growing panorama.

4.3 Comparative Analysis

We compared SIFT and ORB feature detectors to evaluate their performance in the image stitching pipeline.

Table 3: Comparison of Feature Detectors for Image Stitching

Feature Detector	# Keypoints (avg)	Matches	Inliers	Inlier Ratio	Time (s)
SIFT	2547	423	312	73.8%	1.85
ORB	2000	284	187	65.8%	0.63

From Table (3), SIFT detects more keypoints and yields more reliable matches compared to ORB, resulting in a higher inlier ratio with 73.8% vs 65.8% and more robust homography estimation. While SIFT offers superior accuracy, ORB is significantly

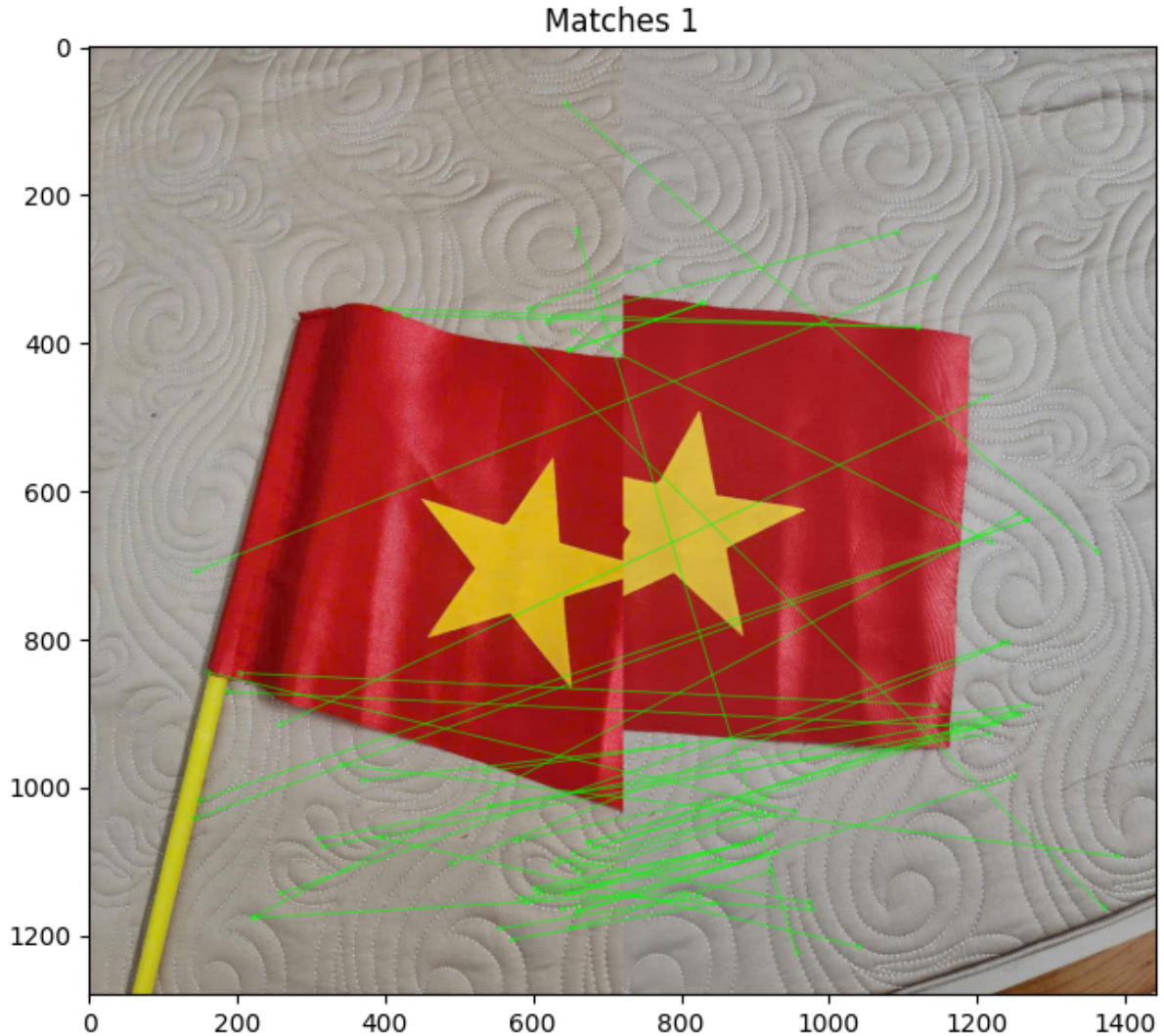


Figure 6: Feature Matches with SIFT feature matches between adjacent images. Green lines indicate inlier matches after RANSAC filtering.

faster—approximately three times quicker—making it more suitable for real-time applications or devices with limited computational resources. In terms of match quality, SIFT provides a more even distribution of matches across the image, whereas ORB tends to concentrate matches in highly textured areas. SIFT also demonstrates better performance under conditions involving significant rotation, scale changes, or perspective distortion. Although both methods are capable of producing acceptable panoramas, SIFT generally achieves higher visual quality with fewer alignment errors and artifacts, especially in scenarios with variable lighting or substantial viewpoint changes.

5 Conclusion

This project has explored three fundamental areas of traditional computer vision: image filtering, 3D reconstruction, and image stitching. Through implementation and comparative analysis, we gained valuable insights into the strengths and limitations of various approaches.

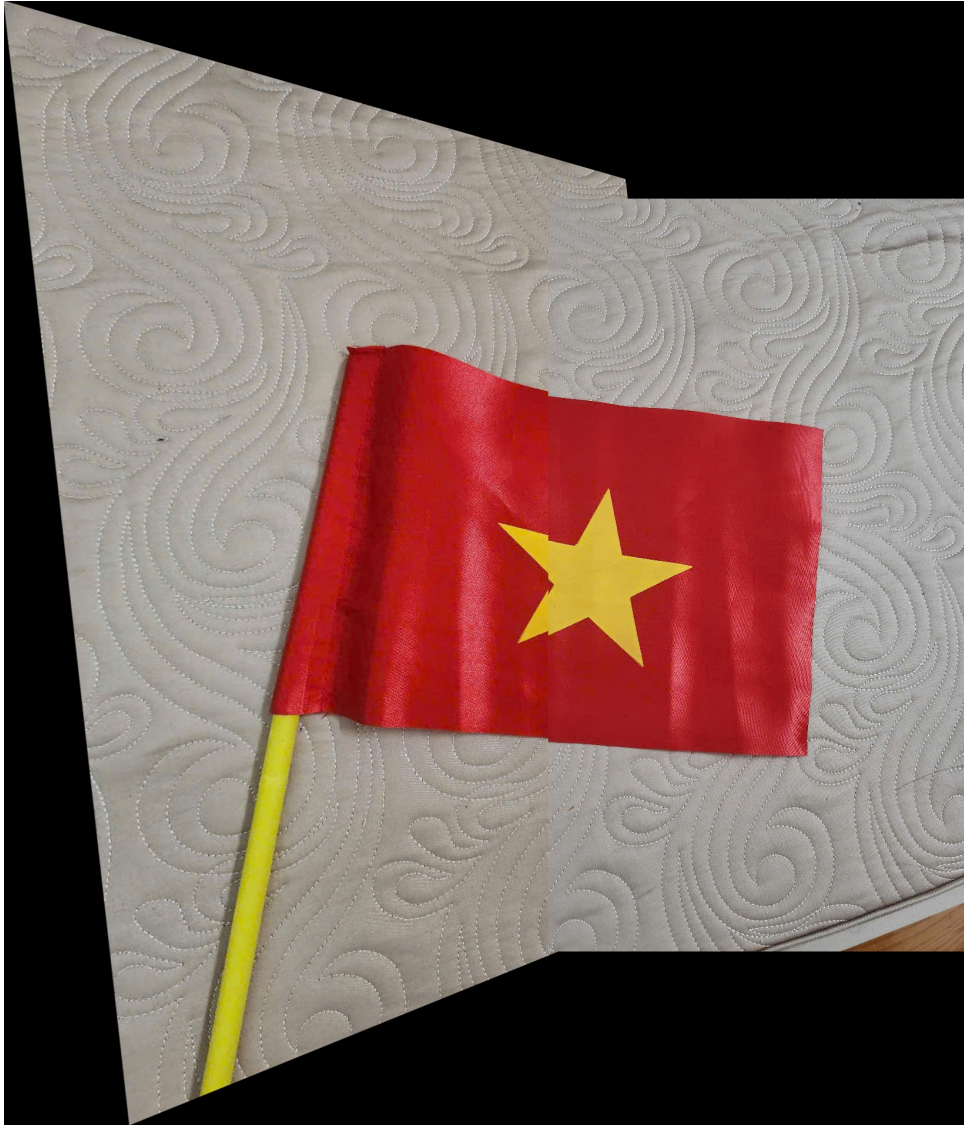


Figure 7: Panorama result created by stitching multiple overlapping images.

For image filtering, we found that different filters excel in specific noise scenarios, Gaussian filters work best for Gaussian noise, median filters are superior for salt-and-pepper noise, and a combination of filtering techniques often yields the best results for complex noise patterns. The trade-off between noise reduction and detail preservation remains a critical consideration, with kernel size being a key parameter affecting this balance.

In 3D reconstruction, Semi-Global Block Matching (SGBM) consistently outperforms Block Matching (BM) in terms of disparity map quality, though at higher computational cost. This highlights the importance of global constraints in stereo matching, particularly for handling textureless regions and occlusions.

For image stitching, SIFT features provide more robust matches compared to ORB, leading to more accurate homography estimation and higher-quality panoramas. However, ORB offers a compelling alternative when processing speed is a priority, demonstrating the practical trade-offs in feature detector selection.

These findings underscore the continued relevance of traditional computer vision techniques, which offer interpretable results, computational efficiency, and strong performance in specific domains. Future work could explore combining traditional methods with modern deep learning approaches to leverage the strengths of both paradigms.

6 References

1. OpenCV Documentation. "Image Filtering." https://docs.opencv.org/4.x/d4/d86/group__imgproc__filter.html
2. Bradski, G., & Kaehler, A. (2008). Learning OpenCV: Computer vision with the OpenCV library. O'Reilly Media.
3. Szeliski, R. (2010). Computer vision: algorithms and applications. Springer Science & Business Media.
4. Hirschmüller, H. (2008). "Stereo processing by semiglobal matching and mutual information." IEEE Transactions on pattern analysis and machine intelligence, 30(2), 328-341.
5. Lowe, D. G. (2004). "Distinctive image features from scale-invariant keypoints." International Journal of Computer Vision, 60(2), 91-110.
6. Rublee, E., Rabaud, V., Konolige, K., & Bradski, G. (2011). "ORB: An efficient alternative to SIFT or SURF." In IEEE International Conference on Computer Vision (ICCV).

A Appendix: Python Code

A.1 Part A: Image Filtering

```
1 import cv2
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from skimage.metrics import peak_signal_noise_ratio as psnr
5 from skimage.metrics import structural_similarity as ssim
6
7 def add_noise(image, noise_type='gaussian', param=0.1):
8     if noise_type == 'gaussian':
9         noise = np.random.normal(0, param,
10                                image.shape).astype(np.float32)
11         noisy_img = np.clip(image + noise, 0,
12                             255).astype(np.uint8)
13     elif noise_type == 'salt_pepper':
14         noisy_img = image.copy()
15         salt = np.random.random(image.shape) < param/2
16         noisy_img[salt] = 255
17         pepper = np.random.random(image.shape) < param/2
18         noisy_img[pepper] = 0
19     elif noise_type == 'speckle':
20         noise = param * np.random.randn(*image.shape)
```

```

19         noisy_img = np.clip(image + image * noise, 0,
20                               255).astype(np.uint8)
21     return noisy_img
22
23 def main():
24     img = cv2.imread('input_image.jpg')
25     gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
26
27     noisy_gaussian = add_noise(gray, 'gaussian', 20)
28     noisy_sp = add_noise(gray, 'salt_pepper', 0.05)
29
30     mean_3x3 = cv2.blur(noisy_gaussian, (3, 3))
31     mean_5x5 = cv2.blur(noisy_gaussian, (5, 5))
32
33     gaussian_3x3 = cv2.GaussianBlur(noisy_gaussian, (3, 3), 0)
34     gaussian_5x5 = cv2.GaussianBlur(noisy_gaussian, (5, 5), 0)
35
36     median_3x3 = cv2.medianBlur(noisy_gaussian, 3)
37     median_5x5 = cv2.medianBlur(noisy_gaussian, 5)
38
39     blurred = cv2.GaussianBlur(noisy_gaussian, (3, 3), 0)
40     laplacian = cv2.Laplacian(blurred, cv2.CV_64F)
41     laplacian_sharpened = np.uint8(np.clip(blurred -
42                                             0.5*laplacian, 0, 255))
43
44 if __name__ == "__main__":
45     main()

```

A.2 Part B: 3D Reconstruction

```

1 import cv2
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import open3d as o3d
5
6 def compute_disparity_map(img_left, img_right, method='sgbm'):
7     gray_left = cv2.cvtColor(img_left, cv2.COLOR_BGR2GRAY)
8     gray_right = cv2.cvtColor(img_right, cv2.COLOR_BGR2GRAY)
9
10    if method == 'bm':
11        stereo = cv2.StereoBM_create(numDisparities=64,
12                                     blockSize=15)
13        disparity = stereo.compute(gray_left, gray_right)
14    else:
15        stereo = cv2.StereoSGBM_create(
16            minDisparity=0,
17            numDisparities=160,
18            blockSize=5,
19            P1=8 * 3 * 5**2,
20            P2=32 * 3 * 5**2,
21            disp12MaxDiff=1,

```

```

21         uniquenessRatio=15,
22         speckleWindowSize=100,
23         speckleRange=32,
24         mode=cv2.STEREO_SGBM_MODE_SGBM_3WAY
25     )
26     disparity = stereo.compute(gray_left, gray_right)
27
28     disparity_normalized = cv2.normalize(disparity, None,
29         alpha=0, beta=255,
30                                         norm_type=cv2.NORM_MINMAX,
31                                         dtype=cv2.CV_8U)
32
33     return disparity, disparity_normalized
34
35 def reconstruct_point_cloud(disparity, img_left, Q):
36     points_3d = cv2.reprojectImageTo3D(disparity, Q)
37     colors = cv2.cvtColor(img_left, cv2.COLOR_BGR2RGB)
38     mask = disparity > 0
39
40     points = points_3d[mask]
41     colors = colors[mask]
42
43     return points, colors
44
45 def main():
46     img_left = cv2.imread('left.jpg')
47     img_right = cv2.imread('right.jpg')
48
49     disparity_bm, disparity_bm_norm =
50         compute_disparity_map(img_left, img_right, 'bm')
51     disparity_sgbm, disparity_sgbm_norm =
52         compute_disparity_map(img_left, img_right, 'sgbm')
53
54 if __name__ == "__main__":
55     main()

```

A.3 Part C: mage Stitching

```

1 import cv2
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 def detect_and_match_features(img1, img2, method='sift'):
6     # Convert to grayscale
7     gray1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
8     gray2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)
9
10    if method == 'sift':
11        # SIFT feature detector
12        detector = cv2.SIFT_create()
13    else:

```

```
14     # ORB feature detector
15     detector = cv2.ORB_create(nfeatures=2000)
16
17     # Detect keypoints and compute descriptors
18     kp1, des1 = detector.detectAndCompute(gray1, None)
19     kp2, des2 = detector.detectAndCompute(gray2, None)
20
21     # Match features
22     if method == 'sift':
23         # FLANN matcher for SIFT
24         FLANN_INDEX_KDTREE = 1
25         index_params = dict(algorithm=FLANN_INDEX_KDTREE,
26                             trees=5)
27         search_params = dict(checks=50)
28         matcher = cv2.FlannBasedMatcher(index_params,
29                                         search_params)
30     else:
31         # Brute-force matcher for ORB
32         matcher = cv2.BFMatcher(cv2.NORM_HAMMING,
33                                 crossCheck=False)
34
35     # Apply ratio test
36     matches = matcher.knnMatch(des1, des2, k=2)
37     good_matches = []
38     for m, n in matches:
39         if m.distance < 0.75 * n.distance:
40             good_matches.append(m)
41
42     return kp1, kp2, good_matches
43
44 def stitch_images(img1, img2, kp1, kp2, matches):
45     # Extract matching points
46     src_pts = np.float32([kp1[m.queryIdx].pt for m in
47                           matches]).reshape(-1, 1, 2)
48     dst_pts = np.float32([kp2[m.trainIdx].pt for m in
49                           matches]).reshape(-1, 1, 2)
50
51     # Find homography
52     H, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC,
53                                 4.0)
54
55     # Warp img1 to img2's perspective
56     h1, w1 = img1.shape[:2]
57     h2, w2 = img2.shape[:2]
58
59     # Calculate the size of the panorama
60     pts = np.float32([[0, 0], [0, h1], [w1, h1], [w1,
61                 0]]).reshape(-1, 1, 2)
62     dst = cv2.perspectiveTransform(pts, H)
```



```
57     dst_corners = np.concatenate((dst, np.float32([[0, 0], [0,
58         h2], [w2, h2], [w2, 0]]).reshape(-1, 1, 2)))
59     [xmin, ymin] = np.int32(dst_corners.min(axis=0).ravel() -
60         0.5)
61     [xmax, ymax] = np.int32(dst_corners.max(axis=0).ravel() +
62         0.5)
63
64     # Translation to keep the entire panorama in frame
65     t = [-xmin, -ymin]
66     Ht = np.array([[1, 0, t[0]], [0, 1, t[1]], [0, 0, 1]])
67
68     # Warp img1
69     result = cv2.warpPerspective(img1, Ht.dot(H), (xmax-xmin,
70         ymax-ymin))
71
72     # Copy img2 to the result
73     result[t[1]:h2+t[1], t[0]:w2+t[0]] = img2
74
75     return result
76
77 def main():
78     # Load images
79     images = []
80     for i in range(4):
81         img = cv2.imread(f'image_{i+1}.jpg')
82         if img is not None:
83             images.append(img)
84
85     # Stitch images
86     result = images[0]
87     for i in range(1, len(images)):
88         kp1, kp2, matches = detect_and_match_features(result,
89             images[i], 'sift')
90         result = stitch_images(result, images[i], kp1, kp2,
91             matches)
92
93     # Display and save result
94     cv2.imwrite('panorama.jpg', result)
95
96 if __name__ == "__main__":
97     main()
```